SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection. **Answer:** Testing should be performed as a systematic engineering activity instead of random testing. The team should follow requirement analysis, test planning, test case design, execution, defect reporting, and regression testing. Techniques such as equivalence partitioning, boundary value analysis, decision tables, and state transition testing should be used. Structured test design improves test coverage, consistency, defect detection, and overall software quality.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples. **Answer:** A well-defined testing process provides a consistent method for planning, designing, executing, and reporting tests. For a banking system, requirements should be analyzed and test cases should cover login, balance checking, fund transfer, withdrawal, and transaction failure conditions. Standardized testing reduces missed defects, improves coverage, supports regression testing, and increases reliability.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3

	Answer: Testing should be performed throughout the SDLC. Requirements are reviewed first, followed by test planning and test design. Unit testing checks individual modules, integration testing checks interactions, system testing checks the complete application, and acceptance testing validates user requirements. Following this structured process helps identify defects early and ensures better quality and reliability.	
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples. **Answer:** An **error** is a human mistake made during development. A **defect** is the fault introduced into the software because of that error. A **failure** occurs when the defect executes and produces an incorrect result. For example, a developer writes an incorrect password condition (error), which creates incorrect login logic (defect), and the system accepts an invalid password (failure). Thus, Error → Defect → Failure.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example. **Answer:** The **early testing principle** should be applied. Testing activities should begin from the requirements and design phases rather than waiting until coding is completed. Early reviews can identify ambiguous or incorrect requirements. For example, detecting an incorrect transaction limit during requirement review is much cheaper than correcting it after deployment. Early defect detection reduces cost and improves software quality.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement. **Answer:** Testers should review requirements for ambiguity, completeness, consistency, and testability. They can identify missing conditions and unclear statements before development starts. For example, if a healthcare requirement does not clearly define acceptable patient-data formats, testers can request clarification. Early tester involvement prevents defects, reduces rework, and improves software quality.	3

7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues. **Answer:** Defects can originate during requirements, design, coding, integration, or deployment. In this case, unclear requirements can cause developers to implement incorrect salary calculations. Clear, complete, and testable requirements should be prepared and reviewed by developers, testers, and stakeholders. Requirement reviews and early validation help prevent defects from reaching later stages.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue. **Answer:** This can be classified as a **boundary-related defect** caused by insufficient testing of extreme input values. Boundary Value Analysis tests minimum, maximum, just below, and just above valid limits. For example, if an input accepts values from 1 to 100, values such as 0, 1, 2, 99, 100, and 101 should be tested. This could have detected the crash before release.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality. **Answer:** A defect repository stores information about previous defects, including module, type, severity, root cause, and resolution. Testers can analyze this information to identify recurring defect patterns and create additional test cases. For example, if payment defects repeatedly occur, more payment-related regression tests can be added. This improves test coverage and prevents recurring defects.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented. **Answer:** This is a **functional defect** because the ATM withdrawal function does not behave according to its requirements. Test cases should include valid withdrawals, insufficient balance, invalid amounts, withdrawal limits, and invalid account conditions. Functional testing,	3

	boundary testing, and regression testing could have detected the defect before deployment.	
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency. **Answer:** Defect clustering means that most defects are often concentrated in a small number of modules. The team should analyze historical defect data and identify high-defect modules. These modules should receive higher testing priority and more detailed test cases. This focuses testing resources on high-risk areas and improves defect detection efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution. **Answer:** This is a **performance defect** because the application becomes slow when subjected to heavy load. It can be further classified according to severity, affected module, environment, and root cause. Load and stress testing can be used to identify the problem. Proper classification helps assign the defect to the correct team and select suitable corrective actions.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency. **Answer:** Historical defect data can be analyzed to identify modules and functions that previously contained defects. Related test cases should be included in the regression suite and executed after every major change. For example, if previous releases had payment-processing defects, those tests should always be included. Reusing historical data improves coverage and reduces the chance of repeated defects while saving testing effort.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps. **Answer:** Testers should participate in requirements reviews, design discussions, planning, and development meetings. They can identify integration risks and clarify expected behavior before coding is completed. Regular communication between testers and developers helps prevent	3

	misunderstandings. Early involvement reduces integration defects, rework, and testing delays.	
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality. **Answer:** The team should perform root-cause analysis for recurring defects and identify why they occur. Code reviews, requirement reviews, developer training, improved coding standards, automated regression testing, and process improvements should be introduced. Instead of fixing only the individual defect, its underlying cause should be removed. This reduces defect recurrence and improves long-term software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability. **Answer:** The team should establish a standardized testing process covering requirements, test planning, test design, execution, defect reporting, retesting, and regression testing. Test case templates and common testing techniques should be used across all modules. Functional, compatibility, performance, and negative testing should be included. Standardization improves consistency, coverage, reliability, and defect detection.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality. **Answer:** Testing identifies failures and reports defects, while debugging locates the cause of the defect and fixes the code. For example, if an ATM deducts the wrong amount, the tester identifies and reports the failure. The developer then examines the transaction logic, finds the coding error, and corrects it. Retesting confirms the fix. Both activities together improve software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled. **Answer:** Cross-browser testing should be performed on major browsers, browser versions, operating systems, and screen sizes. Functional,	3

	compatibility, responsive, and UI testing should be performed. A browser test matrix can be prepared based on customer usage. Automation tools can execute the same tests across multiple browsers. This helps identify compatibility problems and provides consistent user experience.	
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability. **Answer:** The testing team should identify missing input validation and verify both valid and invalid inputs. Equivalence partitioning, boundary value analysis, negative testing, and data validation should be applied. For example, invalid dates, negative quantities, empty fields, and incorrect formats should be tested. Proper validation prevents incorrect data from entering the system and improves reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design. **Answer:** Testing should follow a planned and systematic process. Requirements should be converted into test conditions and structured test cases containing inputs, steps, and expected results. Techniques such as equivalence partitioning, boundary value analysis, decision tables, and state transition testing should be used. Systematic design provides better coverage, repeatability, and defect detection compared with random testing.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency. **Answer:** The tester should verify how data moves between modules and identify incorrect mappings, database operations, synchronization problems, or interface defects. Integration testing should verify communication between modules, while database and data-validation testing should verify stored values. Regression testing should be performed after fixes. These practices ensure accurate and consistent data across modules.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain	3

	the role of testers across SDLC phases. Show how early involvement improves quality. **Answer:** Testers should participate throughout the SDLC. During requirements, they check clarity and testability. During design, they identify risks and prepare test scenarios. During coding, they support unit and integration testing. During system testing, they validate complete functionality, and during maintenance they perform regression testing. Early involvement detects defects sooner, reduces rework, and improves collaboration and quality.	
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures. **Answer:** The testing team should include performance, load, stress, and scalability scenarios in the test plan. Peak concurrent users, large data volumes, response times, and resource usage should be tested. For example, if a system normally supports 1,000 users but may receive 5,000 users during peak time, it should be tested with the expected peak load. Better coverage identifies performance limitations before deployment.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis. **Answer:** A centralized defect repository should record defect ID, title, module, severity, priority, environment, steps to reproduce, expected result, actual result, status, root cause, and resolution. Proper documentation makes defects easier to track and analyze. Historical data can also reveal recurring problems and high-risk modules. Standard templates and regular updates improve defect management.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability. **Answer:** The banking system should follow a structured testing process starting with requirement analysis and risk-based planning. Test cases should cover successful transactions, insufficient balance, invalid account details, transaction limits, duplicate transactions, network failures, and timeouts. Functional, integration, boundary, negative, and regression	3

	testing should be performed. This systematic approach improves coverage, detects defects early, and increases transaction reliability.	
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection. **Answer:** Testing should include normal, invalid, and extreme input conditions. Boundary Value Analysis is useful for testing values at and around limits. For example, if a field accepts values from 1 to 100, values such as 0, 1, 2, 99, 100, and 101 should be tested. Edge-case testing helps identify failures that normal testing may miss and improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes. **Answer:** A common testing process should be applied to every module. Standard test case formats, naming conventions, severity levels, review checklists, execution procedures, and defect reporting formats should be established. Testers should follow the same lifecycle of planning, design, execution, reporting, retesting, and regression. Standardization provides consistent coverage and improves overall quality.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness. **Answer:** Input validation should be tested using valid, invalid, empty, unexpected, boundary, and incorrectly formatted inputs. Negative testing and boundary value analysis should be applied. For example, text fields should be tested with empty values, special characters, excessive length, and invalid formats. Proper validation prevents unexpected behavior and improves application robustness and reliability.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage. **Answer:** Defect leakage may occur because of incomplete requirements, insufficient test coverage, poor test design, lack of regression testing, or inadequate environment testing. The team should perform root-cause analysis and strengthen test planning, reviews, automation, regression	3

	testing, and acceptance testing. High-risk areas should receive priority. These practices detect defects before production and reduce defect leakage.	
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency. **Answer:** The team should introduce testing from the early SDLC stages. Requirements should be reviewed, test scenarios should be prepared early, and unit, integration, system, and acceptance testing should be performed systematically. Static reviews, equivalence partitioning, boundary value analysis, and risk-based testing can detect defects earlier. Early detection reduces rework and maintenance costs, saves time, and improves overall software quality.	3

Code of Conduct:

I G Hemachandran 192421018 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Hemachandran

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand